PharmaLink project

محمد عبد المعز محمد بكري سكشن ٤ مجموعه ٢

محمد الهيثم عمر السليم سكشن ٣ مجموعه ٢

System Request – Pharmacy Locator System

Project Name:

PharmaLink

Project Sponsor:

National Pharmaceutical Association
Name: Ali Abdelsatar
Department: Central Projects Administration
Phone: 010XXXXXXX
Email: AliAbdelsatar@gov.eg

Business Need:

To connect all pharmacies in a central network, enabling users to quickly find and locate medicines. The current system lacks any centralized platform for real-time medicine availability, causing delays for patients especially when seeking urgent or uncommon drugs.

(Pharmacies operate in isolation with no shared inventory visibility, leading to wasted patient time and missed treatments.)

Functionality:

The system should:

- Allow users to search for medicine availability in real time across all registered pharmacies
- Show pharmacy locations and stock levels on a map
- Register and verify pharmacies to ensure credibility
- Provide in-app contact and directions to selected pharmacies
- Support multi-language interface (Arabic and English) and live chat with pharmacists

---

Expected Value:

Tangible:

- Reduce patient medicine search time and unnecessary pharmacy visits
- Increase pharmacy customer reach and revenue through app discoverability

Intangible:

- Improved patient experience and reduced treatment delays
- Better insight for the National Pharmaceutical Association into local medicine supply
- Stronger public trust in registered pharmacies and improved health infrastructure

---

Special Issues / Constraints:

- Must be completed within 6 months
- Must comply with health data protection and privacy regulations
- Budget limit: 450,000 L.E.

Feasibility Study – Pharmacy Locator System

---

1. Technical Feasibility

Assessment:

- **System Requirements:**
  - Real-time medicine availability search across registered pharmacies
  - Location-based map showing pharmacy locations and stock levels
  - Pharmacy registration and verification system
  - In-app contact, directions, and live chat with pharmacists
  - Multi-language support (Arabic and English)
- **Technology Availability:**
  - Flutter framework → cross-platform Android and iOS development is fully supported
  - Mapping APIs and real-time databases → widely available and reliable
  - Backend server and cloud hosting → scalable infrastructure available
- **Expertise and Training:**
  - Development team (2 Flutter devs, 1 backend dev) has the required skills
  - Pharmacy staff onboarding materials will be provided to ensure smooth adoption

Conclusion:
The system is **technically feasible** and can be implemented with current resources.

---

2. Economic Feasibility (With Maintenance)

Assumptions:

- Initial Investment (Year 0): 450,000 L.E.
- Annual Benefits: 180,000 L.E
- Annual Maintenance Cost: 25,000 L.E.
- Project life: 4 years
- Discount rate: 10%

Step 1 – Annual Net Cash Flow

Year	Total Benefit (L.E.)	Maintenance Cost (L.E.)	Net Cash Flow (L.E.)	Cumulative Cash Flow (L.E.)
0	–	–	–450,000	–450,000
1	180,000	25,000	155,000	–295,000
2	180,000	25,000	155,000	–140,000
3	180,000	25,000	155,000	15,000
4	180,000	25,000	155,000	170,000

Step 2 – Break-Even Point (BEP)

BEP=Initial Investment/Annual Net Cash Flow

BEP=2.90 years

✅ Investment is recovered in **Year 3 (around 2 years and 11 months)**.

Step 3 – Return on Investment (ROI)

Step 1 – Total Cost Including Maintenance

- Initial Investment (Year 0): 450,000 L.E.
- Maintenance: 25,000 L.E./year × 4 years = 100,000 L.E.
- **Total Cost:** 450,000 + 100,000 = 550,000 L.E.

Step 2 – Total Benefits

- Total Benefits over 4 years: 180,000 × 4 = 720,000 L.E.

Step 3 – ROI Calculation

$$ROI = \frac{\text{Total Benefits} - \text{Total Costs}}{\text{Total Costs}}$$

ROI= = 31%

Step 4 – Present Value (PV) and NPV

PV = Cash flow amount / (1 + R)^ n ,

where

interest rate = required return

n = number of years in future

R=10

Year	Net Cash Flow	PV (L.E.)
0	–450,000	–450,000
1	155,000	155,000 ÷ 1.1 ≈ 140,909
2	155,000	155,000 ÷ 1.21 ≈ 128,099
3	155,000	155,000 ÷ 1.331 ≈ 116,454
4	155,000	155,000 ÷ 1.4641 ≈ 105,867

NPV:

NPV= −450,000+140,909+128,099+116,454+105,867≈41,329 L.E.

Positive NPV → project is financially feasible; investment is recovered and generates a net benefit over the 4-year period.

---

3. Organizational Feasibility

Assessment:

- **Staff Capability:**
  - Pharmacy staff can be onboarded quickly with provided training materials
  - App is designed for ease of use, requiring minimal technical knowledge from end users
- **Impact on Operations:**
  - Reduces time patients spend searching for medicines across multiple pharmacies

    - Increases pharmacy visibility and customer flow through real-time discovery
    - Gives healthcare agencies better insight into regional medicine supply gaps
- **Management Support:**
    - Project sponsored by the National Pharmaceutical Association → strong government-level support
    - Pharmacy onboarding and staff training plan is included within the project schedule

Conclusion:
The system is **organizationally feasible**.

---

4. Overall Feasibility Summary

Aspect	Feasibility	Notes
Technical	✔ Feasible	Flutter + mapping APIs + cloud backend are all available and proven
Economic	⚖ Borderline	ROI = 31%, NPV positive; project is financially viable
Organizational	✔ Feasible	Pharmacy staff can be trained easily; strong NPA sponsorship ensures adoption

Decision:

- Proceed with system development.
- Monitor **pharmacy onboarding progress and medicine data quality** to ensure NPV continues to grow and ROI is maximized.

Interview Schedule

Name	Position	Purpose of Interview	Meeting
Joey Samwell	Backend Developer	Backend architecture and API design; current challenges and goals	Mon, March 1 9:00–10:00 A.M.
Belal Aziz	Flutter Developer	Mobile UI development and cross-platform concerns; future goals	Mon, March 1 11:00 A.M.–12:00 P.M.

Interviews

1.

Interviewer: Mohamed Alhaitham

Interviewee: Joey Samwell

Type of interviewee: Backend Developer

Purpose of interview:

- To determine the functional and nonfunctional requirements of the PharmaLink backend system
- To identify current technical challenges with the backend architecture
- To specify improvements and optimizations needed for the system
- Get the developer's technical vision and goals for the project

Summary of the interview:

The most common challenges discussed regarding the backend system:

- Scalability issues when handling concurrent pharmacy requests
- Need for a reliable real-time database synchronization mechanism
- API design and integration with the Flutter frontend

Open items:

- Set up a follow-up interview to discuss database schema design
- Complete the interview to determine deployment and hosting strategy

Detailed notes:

When I interviewed Joey Samwell, the following happened:

What are the main backend challenges you foresee in building PharmaLink?

- He said: “the biggest challenge is ensuring real-time inventory updates across all registered pharmacies without latency, especially during peak usage hours”

What technology stack do you recommend for the backend?

- He said: “I suggest using Node.js with a cloud-hosted database like Firebase or Supabase, as they support real-time data sync out of the box and scale well with Flutter apps”

Have you encountered any concerns about data privacy compliance?

- He said: “yes, we need to ensure all patient search data is anonymized and that the system complies with health data protection regulations from day one”

2.

Interviewer: Mohamed Abdelmoez

Interviewee: Belal Aziz

Type of interviewee: Flutter Developer

Purpose of interview:

- To determine the UI/UX and functional requirements from a mobile development perspective
- To identify cross-platform compatibility challenges
- To specify improvements needed for the mobile interface
- Get the developer's opinion on the app's design and architecture

Summary of the interview:

The most common challenges discussed regarding the Flutter frontend:

- Ensuring smooth map rendering and real-time location updates
- Supporting both Arabic and English with proper RTL layout
- Optimizing app performance on lower-end Android devices

Open items:

- Set up a follow-up interview to finalize UI component structure
- Complete the interview to determine offline mode requirements

Detailed notes:

When I interviewed Belal Aziz, the following happened:

What are the main frontend challenges you foresee in building PharmaLink?

- He said: "implementing a smooth and accurate map view that updates pharmacy stock in real-time is the most technically demanding part of the UI"

How do you plan to handle multi-language support?

- He said: "Flutter has good built-in localization support, but we need to carefully design layouts that work for both Arabic RTL and English LTR without breaking the UI"

Have you encountered any performance concerns?

- He said: "sometimes, when loading many pharmacy markers on the map simultaneously, the app can slow down, so we need to implement clustering and lazy loading"

Functional & Non-Functional Requirements

Project: PharmaLink – Pharmacy Locator System
Interviewer: Mohamed Abdelmoez
Interviewee: Belal Aziz
Role: Flutter Developer

1. Functional Requirements

The following functional requirements were derived from Belal Aziz's interview responses regarding the PharmaLink Flutter mobile application.

ID	Requirement	Source
FR-01	The system shall display an interactive map showing all registered pharmacy locations and their real-time stock levels.	Belal Aziz – Q1: map view that updates pharmacy stock in real-time
FR-02	The system shall update pharmacy stock information on the map in real time as inventory changes occur.	Belal Aziz – Q1: updates pharmacy stock in real-time
FR-03	The system shall implement marker clustering on the map so that nearby pharmacies are grouped when zoomed out.	Belal Aziz – Q3: implement clustering and lazy loading
FR-04	The system shall implement lazy loading for pharmacy markers to avoid loading all locations at once.	Belal Aziz – Q3: implement clustering and lazy loading
FR-05	The system shall support a full Arabic (RTL) interface as an alternative to the English (LTR) interface.	Belal Aziz – Q2: layouts that work for both Arabic RTL and English LTR
FR-06	The system shall allow users to switch between Arabic and English languages from within the app settings.	Belal Aziz – Q2: Flutter localization support
FR-07	The system shall allow users to search for medicine availability across all registered pharmacies in real time.	Belal Aziz – Purpose of interview / general
FR-08	The system shall provide in-app directions and contact options to guide the user to their selected pharmacy.	Belal Aziz – Purpose of interview / general

2. Non-Functional Requirements

2.1 Performance

- **The map shall load and display up to 100 pharmacy markers within 3 seconds on a standard 4G connection. (Source: Belal Aziz – Q3: app can slow down when loading many markers)**
- **Real-time stock updates on the map shall reflect inventory changes with a maximum delay of 2 seconds. (Source: Belal Aziz – Q1: smooth and accurate map view that updates in real-time)**

2.2 Usability

- **The app layout shall fully adapt to RTL alignment when Arabic is selected, without misaligning any UI components. (Source: Belal Aziz – Q2: without breaking the UI)**
- **The app interface shall be operable by end users (patients) with no prior technical training required. (Source: Belal Aziz – Purpose of interview)**
- **Language switching shall take effect immediately without requiring an app restart. (Source: Belal Aziz – Q2: Flutter localization support)**

2.3 Compatibility & Portability

- **The Flutter application shall run on both Android and iOS platforms from a single shared codebase. (Source: Belal Aziz – General: Flutter cross-platform)**
- **The application shall be optimized to run smoothly on lower-end Android devices with limited RAM and CPU. (Source: Belal Aziz – Q3: performance concerns on lower-end devices)**

2.4 Scalability

- **The app's map component shall handle an increasing number of pharmacy markers (500+) without degrading UI responsiveness. (Source: Belal Aziz – Q3: abundance of markers slows the app)**

2.5 Maintainability

- **The Flutter codebase shall follow a modular architecture (e.g., feature-based folders) to allow independent updates to UI components. (Source: Belal Aziz – General)**
- **The localization strings shall be stored in separate resource files to allow easy addition of new languages in the future. (Source: Belal Aziz – Q2: Flutter built-in localization support)**

Functional & Non-Functional Requirements

Project: **PharmaLink – Pharmacy Locator System**
Interviewer: **Mohamed Alhaitham**
Interviewee: **Joey Samwell**
Role: **Backend Developer**

1. Functional Requirements

The following functional requirements were derived from Joey Samwell's interview responses regarding the PharmaLink backend system.

ID	Requirement	Source
FR-01	The system shall synchronize pharmacy inventory data in real time across all registered pharmacies with no noticeable latency.	Joey Samwell – Q1: real-time inventory updates without latency
FR-02	The system shall maintain reliable real-time data synchronization especially during peak usage hours when concurrent requests are high.	Joey Samwell – Q1: during peak usage hours
FR-03	The system shall use Node.js as the backend runtime environment to handle API requests from the Flutter frontend.	Joey Samwell – Q2: using Node.js
FR-04	The system shall use a cloud-hosted real-time database (Firebase or Supabase) to store and sync pharmacy inventory data.	Joey Samwell – Q2: cloud-hosted database like Firebase or Supabase
FR-05	The system shall expose RESTful APIs that allow the Flutter frontend to query pharmacy stock and location data.	Joey Samwell – Q2: scale well with Flutter apps
FR-06	The system shall anonymize all patient search queries before storing or processing them.	Joey Samwell – Q3: patient search data is anonymized
FR-07	The system shall enforce compliance with applicable health data protection and privacy regulations from the initial release.	Joey Samwell – Q3: complies with health data protection regulations from day one
FR-08	The system shall support pharmacy registration and verification through a backend admin interface.	Joey Samwell – Purpose of interview / general

2. Non-Functional Requirements

2.1 Performance

- **The system shall synchronize pharmacy inventory updates with a maximum delay of 2 seconds under normal network conditions. (Source: Joey Samwell – Q1: real-time inventory updates without latency)**
- **The backend API shall respond to frontend requests within 1 second under standard load. (Source: Joey Samwell – Q1: without latency during peak usage hours)**

2.2 Scalability

- **The backend shall scale horizontally to handle concurrent requests from all registered pharmacies simultaneously without performance degradation. (Source: Joey Samwell – Q1: peak usage hours / Summary: scalability issues)**
- **The cloud infrastructure shall support auto-scaling during periods of high demand without requiring manual intervention. (Source: Joey Samwell – Q2: cloud-hosted database that scales well)**

2.3 Security & Privacy

- **All patient search data shall be anonymized before it is stored in or transmitted through the system. (Source: Joey Samwell – Q3: patient search data is anonymized)**
- **The system shall comply with applicable national and international health data protection regulations from the first release. (Source: Joey Samwell – Q3: from day one)**
- **Only verified and registered pharmacies shall be authorized to update stock information via the backend. (Source: Joey Samwell – General)**

2.4 Reliability

- **The system shall maintain a minimum uptime of 99.5% to ensure continuous availability for patients and pharmacies. (Source: Joey Samwell – Q1: real-time sync without latency)**
- **The real-time database shall implement automatic failover mechanisms to prevent data loss during server outages. (Source: Joey Samwell – Q2: Firebase or Supabase support real-time data sync out of the box)**

2.5 Maintainability

- **The backend API shall be fully documented using standard conventions (e.g., OpenAPI/Swagger) to facilitate future development and maintenance. (Source: Joey Samwell – Q2: technology stack recommendation)**
- **The backend codebase shall follow a modular architecture to allow independent updates and testing of individual system components. (Source: Joey Samwell – General / Summary: API design)**

2.6 Interoperability

- **The backend shall provide a well-defined API contract so that the Flutter frontend can integrate without requiring backend code changes. (Source: Joey Samwell – Q2: scale well with Flutter apps)**
- **The system shall support integration with third-party mapping services for pharmacy location data. (Source: Joey Samwell – General)**

Use Case: Search Medicine Availability

- **Use Case Name:**

  Search Medicine Availability

- **Use Case ID:**

  UC-01

- **Description:**

  A registered or guest user searches for the real-time availability of a specific medicine across all registered pharmacies on PharmaLink. The system returns a list of nearby pharmacies that have the medicine in stock, along with their locations on a map.

- **Priority:**

  High

- **Actor:**

  Patient / General User

- **Trigger:**

  User enters a medicine name in the search bar and clicks the "Search" button.

- **Preconditions:**

  - The user knows the name of the medicine they are looking for.
  - The user face difficulty finding the medicine.
  - The pharmacy owner does not already have a registered account.

- **Normal Scenario (Step-by-Step):**

  1. User opens the PharmaLink app.
  2. User types the medicine name in the search bar, e.g., "Paracetamol 500mg".
  3. User selects a search radius, e.g., 5 km.
  4. User clicks the Search button.
  5. System queries the pharmacy database for real-time stock matching the medicine.
  6. System returns results and displays them in a list and on a map.
  7. User taps on a pharmacy pin on the map to view details:
     - Pharmacy name and address
     - Contact phone number
  8. User selects "Get Directions".
  9. System launches navigation to the selected pharmacy.

- **Alternative Scenario:**
  - **A1: Medicine Not Found in Any Nearby Pharmacy**

● User searches for "Morphine 10mg".

● No registered pharmacy within the selected radius has it in stock.

● System shows: "No pharmacies found with this medicine nearby. Try expanding your search radius."

● User increases the radius to 20 km and retries — results are found.

  - **A2: Misspelled Medicine Name**

● User types "Paractem" instead of "Paracetamol".

● System uses fuzzy matching and suggests: "Did you mean Paracetamol?"

● User confirms the suggestion.

● System displays results normally.

- **Exception Scenario:**

E1: Database Connection Timeout

● User submits the search.

● System fails to connect to the pharmacy database within 10 seconds.

● Message: "Search failed. Please check your connection and try again."

- **Postconditions:**

● User sees a list and map of pharmacies stocking the requested medicine.

● User can proceed to contact or navigate to a chosen pharmacy.

- **Inputs:**

Input	Source
Medicine name (text)	User
Search radius (km)	User
User location (GPS)	Device / User

- **Outputs:**

Output	Destination
List of pharmacies with stock	User (App Screen)
Map with pharmacy location pins	User (App Screen)
Pharmacy contact and directions	User (App Screen)

By

- Mohamed Elhaysam Omar (محمد الهيثم عمر سكشن 3 مجموعة 2)
- Mohamed Abdelmoez (محمد عبدالمعز محمد بكري سكشن 4 مجموعة 2)

Use Case: Live Chat with Pharmacist

- **Use Case Name:**

  Live Chat with Pharmacist

- **Use Case ID:**

  UC-02

- **Description:**

  A user who has found a pharmacy on PharmaLink wants to ask the pharmacist a question directly through the app, such as asking about dosage or availability of a specific variant without needing to call or visit the pharmacy.

- **Priority:**

  Medium

- **Actor:**

  Patient / General User

- **Trigger:**

  User clicks the "Chat with Pharmacist" button on a pharmacy's profile page.

- **Preconditions:**

  - User has selected a specific pharmacy from the search results.
  - The selected pharmacy has the live chat feature enabled.

- **Normal Scenario (Step-by-Step):**

  1. User searches for a medicine and opens a pharmacy profile.
  2. User clicks "Chat with Pharmacist".
  3. System opens a chat window connected to that pharmacy.
  4. System notifies the pharmacist of a new incoming chat.
  5. User types a question.
  6. User clicks Send.
  7. System delivers the message to the pharmacist in real time.
  8. Pharmacist reads the message and types a reply.
  9. System delivers the pharmacist's reply to the user.
  10. User reads the reply and either continues the conversation or closes the chat.
  11. System saves the chat history in the user's account for future reference.

- **Alternative Scenario:**
  - **A1: Pharmacist is Offline**

● User clicks "Chat with Pharmacist".

● System detects the pharmacist is currently offline.

● System shows: "Pharmacist is currently unavailable. You can leave a message and they will reply shortly."

● User leaves a message.

● System notifies the pharmacist when they come back online.

  - **A2: User Sends an Empty Message**

● User clicks Send without typing anything.

● System shows: "Message cannot be empty."

● User types a message and resends.

- **Exception Scenario:**

E1: Chat Connection Lost

● User is in the middle of a conversation.

● The internet connection drops.

● System shows: "Connection lost. Reconnecting..."

● Once reconnected, previous messages are reloaded and the chat continues.

- **Postconditions:**

● Chat history is stored in the user's account.

● User receives an answer to their question without visiting the pharmacy.

- **Inputs:**

Input	**Source**
Selected pharmacy	User
Chat message (text)	User
Pharmacist reply (text)	Pharmacist

- **Outputs:**

Output	**Destination**
Delivered message	Pharmacist (Chat Window)
Pharmacist reply	User (Chat Window)
Chat history record	Database
Offline / error notifications	User (App Screen)

By

- Mohamed Elhaysam Omar (محمد الهيثم عمر سليم سكشن 3 مجموعة 2)
- Mohamed Abdelmoez (محمد عبدالمعز محمد بكري سكشن 4 مجموعة 2)

Use Case: Switch App Language

- **Use Case Name:**

  Switch App Language

- **Use Case ID:**

  UC-03

- **Description:**

  A user wants to change the display language of the PharmaLink application between Arabic and English. The system updates the entire interface including menus, buttons, labels, and search results to reflect the selected language immediately.

- **Priority:**

  High

- **Actor:**

  Patient / General User

- **Trigger:**

  User clicks the language toggle button (EN / ع) found in the app settings or the top navigation bar.

- **Preconditions:**

  ● User has the PharmaLink app open.

  ● The app is currently displayed in a language different from the one the user wants to switch to.

- **Normal Scenario (Step-by-Step):**

  1. User opens the PharmaLink app, which is currently displayed in English.
  2. User clicks the language toggle button "ع" in the top navigation bar.
  3. System detects the requested language change: English → Arabic.
  4. System reloads the interface in Arabic, including:
     - All menus and navigation labels
     - Search bar placeholder text
     - Buttons and confirmation messages
     - Layout direction switches from left-to-right (LTR) to right-to-left (RTL)
  5. System saves the selected language preference to the user's account.
  6. User sees the full app displayed in Arabic.

7. All future sessions open in Arabic until the user switches again.

- **Alternative Scenario:**
  - **A1: User Switches Back to English**
  - App is currently in Arabic.
  - User clicks the language toggle button "EN".
  - System reloads the interface in English with LTR layout.
  - Language preference is updated to English in the user's account.
  - **A2: User is Not Logged In**
  - User switches the language without being logged in.
  - System applies the language change for the current session only.
  - System shows: "Log in to save your language preference permanently."
  - On the next session, the app reverts to the default language (English).

- **Exception Scenario:**

  E1: Language Pack Fails to Load
  - User clicks the language toggle button.
  - System attempts to load the Arabic language pack but encounters a network error.
  - Message: "Could not load language. Please check your connection and try again."
  - App remains in the current language until the issue is resolved.

- **Postconditions:**
  - The entire app interface is displayed in the selected language.
  - The language preference is saved in the user's account for all future sessions.

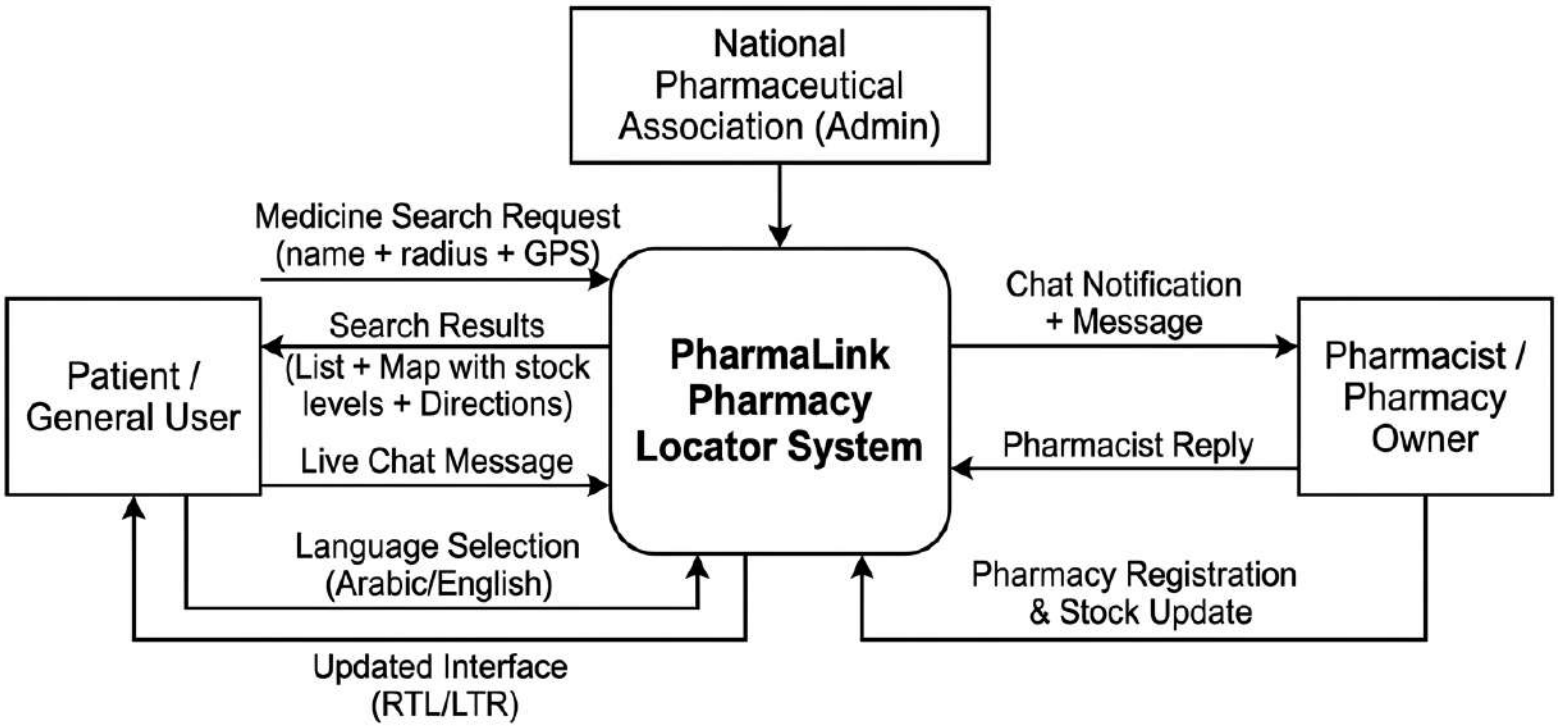
- **Inputs:**

Input	Source
Selected language (Arabic / English)	User
User account ID	System

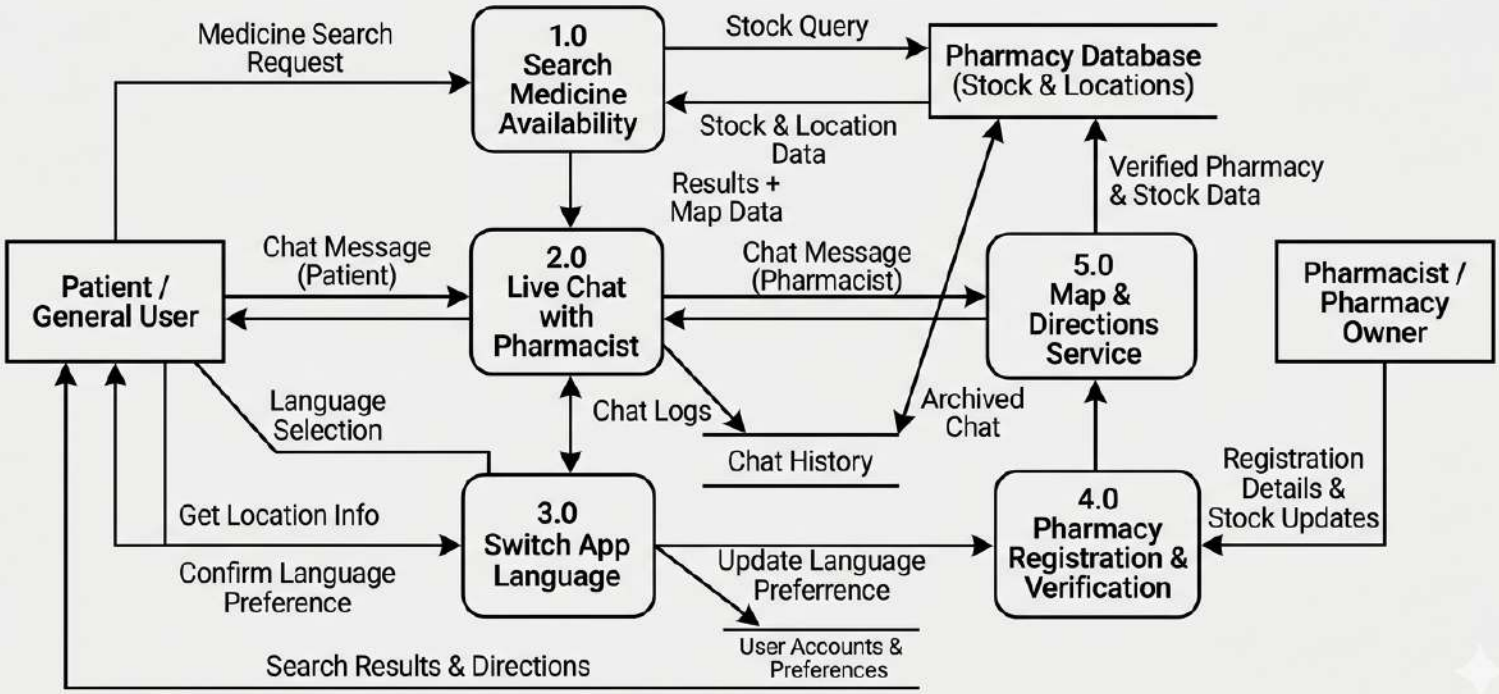
- **Outputs:**

Output	Destination
App interface in selected language	User (App Screen)
RTL / LTR layout adjustment	User (App Screen)
Saved language preference	Database
Error message (if language fails to load)	User (App Screen)

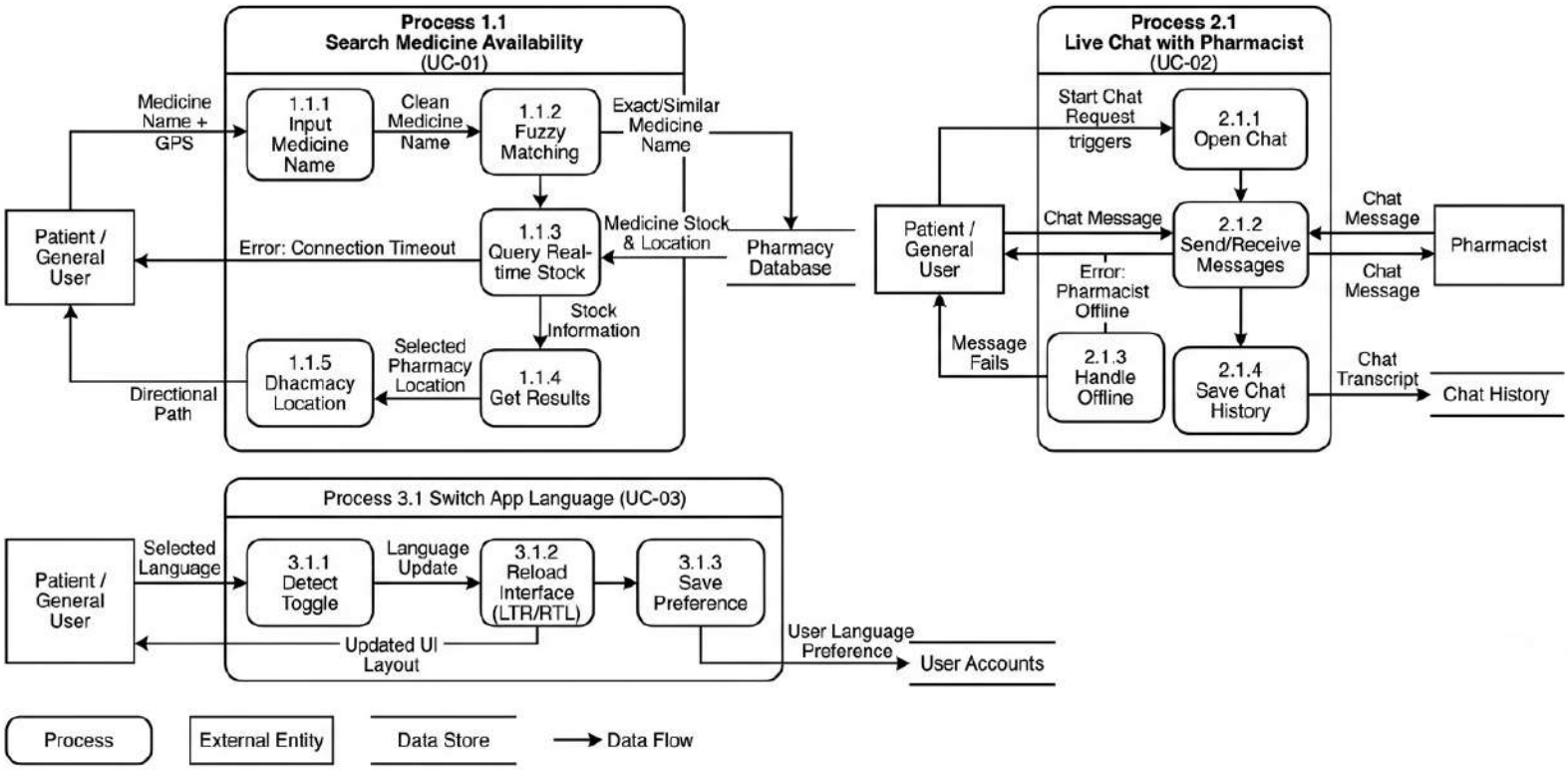
Context Diagram - PharmaLink Pharmacy Locator System



Level 0 DFD - PharmaLink Pharmacy Locator System



Level 1 DFD - PharmaLink (Detailed Core Processes)



Hardware and Software Specification

	Standard Application Server	Standard Database Server
Operating System	• Windows / Linux / Mac (Backend Server)	• Windows / Linux / Mac
Special Software	• Flutter Framework • Firebase / Node.js (Real-time Backend & • Google Maps API	• MySQL or Firestore Live Chat) (Real-time Database)
Hardware	• Google Maps API • Sufficient RAM & CPU for real-time queries • Intel Core CPU	• 2-TB Disk Drive • Intel Core CPU • RAID Level 6
Network	• High-speed Internet • 10-GB Ethernet	• High-speed Internet • 10-GB Ethernet